

Customer Support Q&A API

A production-oriented Node.js API that authenticates support requests, grounds OpenAI answers in business policy, and returns a stable JSON contract with operational safeguards.

DELIVERY	PROVIDER	SECURITY	QUALITY
Authenticated REST API	OpenAI Responses API	Strong bearer auth + limits	18 tests + multi-version CI

Business challenge

Support teams need fast, consistent answers without exposing a paid AI endpoint directly to callers. The service must validate requests, protect credentials, control provider cost, preserve a predictable error contract, and make readiness observable.

Implemented solution

The API accepts an authenticated customer-support question, validates and normalizes the payload, combines it with a maintained business-policy context, sends a bounded Responses API request with store: false, and returns only a completed answer with model and token-usage metadata.

Request-to-answer flow

1. PROTECT	2. VALIDATE	3. ANSWER	4. NORMALIZE
Security headers, process-level IP rate limit, and timing-safe authentication with an independent 32-byte secret.	JSON parser with 32 KB limit, object schema, string type, trim, and question length bounds.	Fixed instructions, store: false, timeout, bounded retries, and completed-response enforcement.	Stable success shape and public error codes; incomplete output and provider details stay server-side.

Grounding approach

Business facts live in a separate support-context module. The prompt instructs the model to answer only from that context and treats the customer question as untrusted data. This creates a clear maintenance point for approved policy content and a direct path to a retrieval layer when the knowledge base grows.

API contract

Successful responses contain answer, model, and normalized usage fields. Malformed input, missing authentication, rate limits, provider failures, readiness issues, and unknown routes use documented status codes and stable JSON error identifiers.

A backend built for predictable behavior

The portfolio project emphasizes the controls that turn a provider call into an operable API service.

Security controls	Provider resilience
- Validated independent 32-byte inbound secret - Timing-safe bearer-token comparison - Helmet security headers - Explicit proxy-trust configuration	- SDK request timeout and bounded retries - Incomplete and refusal responses rejected - Stable mapping of provider errors - Graceful process shutdown

Operational endpoints

- GET /health/live confirms that the HTTP process is running.
- GET /health/ready confirms required secret configuration.
- POST /support/ask is the authenticated, rate-limited paid endpoint.
- SIGINT and SIGTERM trigger graceful server shutdown.

Verification evidence

18 unit and HTTP integration tests cover credentials, validation, prompt boundaries, store: false, completed-response handling, real SDK errors, authentication, health endpoints, malformed and oversized JSON, rate limiting, and stable errors. CI also validates OpenAPI and the quality-evaluation dataset on Node.js 20, 22, and 24.

Data boundary

The complete normalized question is sent to OpenAI. Responses application-state storage is disabled with store: false; separate provider abuse-monitoring retention remains subject to the OpenAI project's data controls. Production intake requires data minimization and an approved privacy policy.

Production rollout

Replace the bundled policy context with approved business content, move secrets to a managed store, configure TLS and proxy trust, add centralized logs and cost monitoring, choose a distributed rate-limit store for multiple instances, and run representative answer-quality and prompt-injection evaluations.

Business value

The service provides a secure integration layer between customer-facing channels and OpenAI, keeping authentication, validation, cost controls, provider behavior, and public errors consistent across future web, helpdesk, or CRM clients.